COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/15/26	**DATE SUBMITTED:**	10/15/26
NAME:	Reyes, Cyrill Joferson B. Silvoza, Keith Nicole B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Factorial Calculation	The function directly calls itself with a reduced parameter (e.g., factorial(n - 1)) until reaching a base case (n <= 1), solving the main problem through stacked function evaluations.
Brute Force	Linear Search	The algorithm evaluates every single element or candidate sequentially from start to end without using optimizations, heuristics, or search space reduction.
Backtracking	N-Queens Problem	The algorithm incrementally builds candidate solutions and immediately abandons ("backtracks") a branch whenever a state violates constraints (e.g., two queens attacking each other).
Greedy	Fractional Knapsack	At each decision step, the algorithm selects the choice that appears locally optimal (e.g., taking the item with the highest value-to-weight ratio) without undoing past choices.
Divide-and-Conquer	Merge Sort	The problem is split into independent subproblems (**Divide**), solved recursively (**Conquer**), and

		their results are merged back together (**Combine**).

Table 12-1. Algorithmic Strategies and Examples

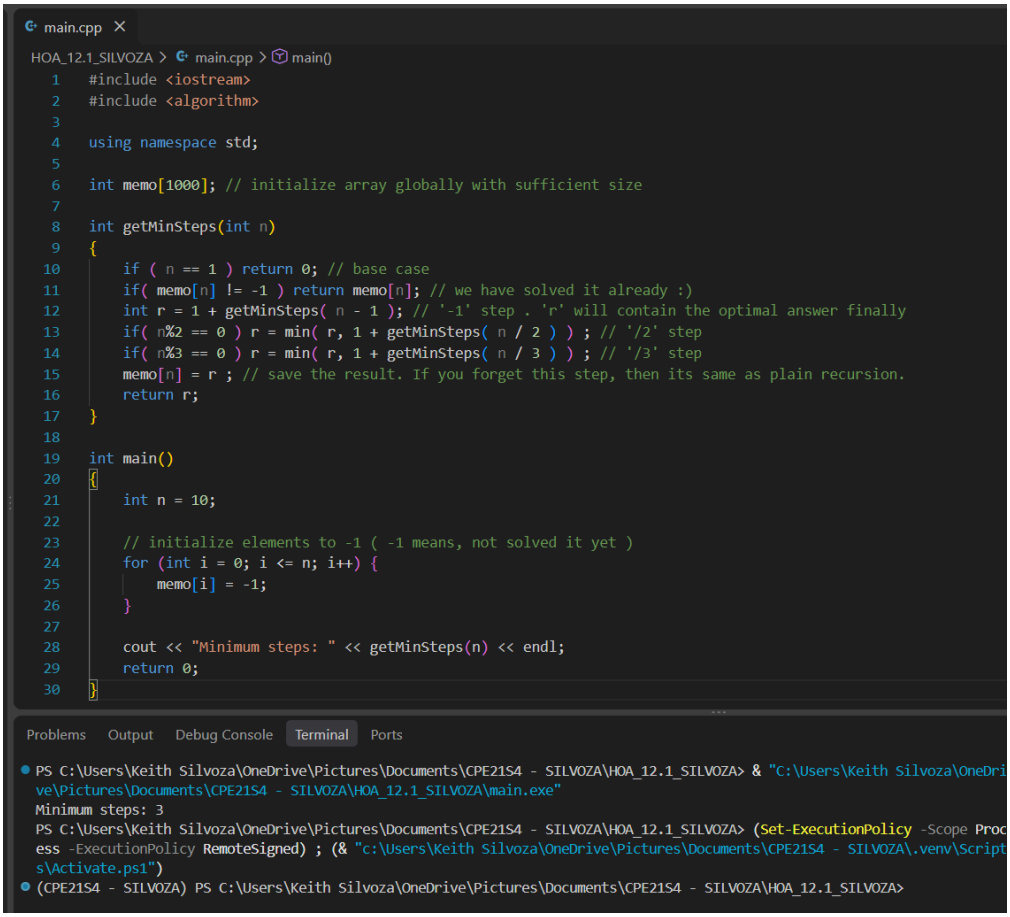
Screenshot	
Analysis	Memoization solves the problem by starting from n and breaking it down recursively. By storing already calculated answers in an array (memo[]), it avoids re-computing overlapping subproblems, reducing time complexity from exponential $O(3^n)$ to linear $O(n)$.

Table 12-2. Memoization Implementation

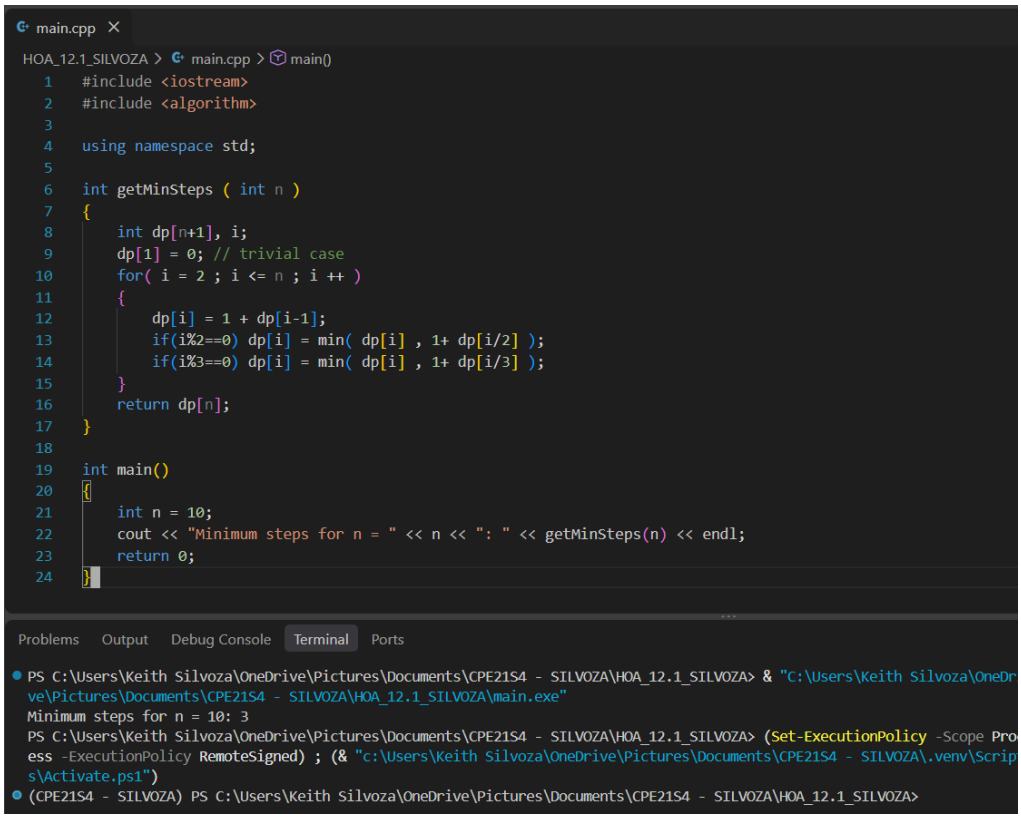
Screenshot	
Analysis	The bottom-up approach starts from the base case ($i = 2$) and builds up to n iteratively using a loop. It runs in $O(n)$ time complexity and uses memory efficiently without the call-stack overhead or risk of stack overflow seen in recursive calls.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

REYES

PSEUDOCODE

Algorithm: countPaths(mat, m, n, cost)
Input: 2D Matrix mat, current cell indices (m, n), remaining cost

Output: Integer count of valid paths

1. IF m == 0 AND n == 0 THEN
   IF mat[0][0] == cost THEN
      RETURN 1
   ELSE
      RETURN 0
   END IF
2. IF m == 0 THEN

RETURN countPaths(mat, 0, n - 1, cost - mat[0][n])
3. IF n == 0 THEN
RETURN countPaths(mat, m - 1, 0, cost - mat[m][0])
4. RETURN countPaths(mat, m - 1, n, cost - mat[m][n]) +
countPaths(mat, m, n - 1, cost - mat[m][n])

Solving by hand

$(0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (3, 3)$

Sum: $4 + 7 + 1 + 3 + 1 + 2 + 3 = 21$

Code

HOA 12p1 > supplementary.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <map>

using namespace std;

int countPathsDP(const vector<vector<int>>& mat, int m, int n, int cost, map<string, int>& memo) {
    if (m == 0 && n == 0) {
        return (mat[0][0] == cost) ? 1 : 0;
    }

    if (cost < 0) return 0;

    string key = to_string(m) + "," + to_string(n) + "," + to_string(cost);

    if (memo.find(key) != memo.end()) {
        return memo[key];
    }

    int totalPaths = 0;

    if (m == 0) {
        totalPaths = countPathsDP(mat, 0, n - 1, cost - mat[m][n], memo);
    } else if (n == 0) {
        totalPaths = countPathsDP(mat, m - 1, 0, cost - mat[m][n], memo);
    } else {
        totalPaths = countPathsDP(mat, m - 1, n, cost - mat[m][n], memo) +
                     countPathsDP(mat, m, n - 1, cost - mat[m][n], memo);
    }
```

```
    memo[key] = totalPaths;
    return memo[key];
}

int main() {
    vector<vector<int>> mat = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int targetCost = 25;
    int M = mat.size();
    int N = mat[0].size();

    map<string, int> memo;
    int totalPaths = countPathsDP(mat, M - 1, N - 1, targetCost, memo);

    cout << "Matrix Size: " << M << "x" << N << endl;
    cout << "Target Cost: " << targetCost << endl;
    cout << "Total paths with cost " << targetCost << " : " << totalPaths << endl;

    return 0;
}
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes> cd "c:\Users\Jofjof\OneDrive\Docume
}
Matrix Size: 4x4
Target Cost: 25
Total paths with cost 25 : 0
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\HOA 12p1> (Set-ExecutionPolicy -Scop
ctivate.ps1")
(venv) PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\HOA 12p1>
```

Analysis

Core Goal: The program counts how many ways you can travel from the start cell $(0, 0)$ to the end cell $(M - 1, N - 1)$ moving only Right or Down so that the sum of the numbers along the path equals a target cost.

How the Recursion Works: Moving backward from the destination cell, the algorithm subtracts the current cell's value from the required target cost and recursively checks the paths coming from the cell above $(m - 1, n)$ and the cell to the left $(m, n - 1)$.

Why Memoization is Needed: Plain recursion visits the same cells with the same remaining cost multiple times, causing unnecessary repeated work. Storing computed results in a lookup memory (memo) eliminates redundant calculations.

Result for Target Cost 25: For the given 4×4 grid, the program returns 0 paths because no combination of Right/Down moves produces a sum of exactly 25.

SILVOZA

PSEUDOCODE

```
FUNCTION countPaths(matrix, i, j, cost):
    IF i is out of bounds OR j is out of bounds OR cost < 0 THEN
        RETURN 0

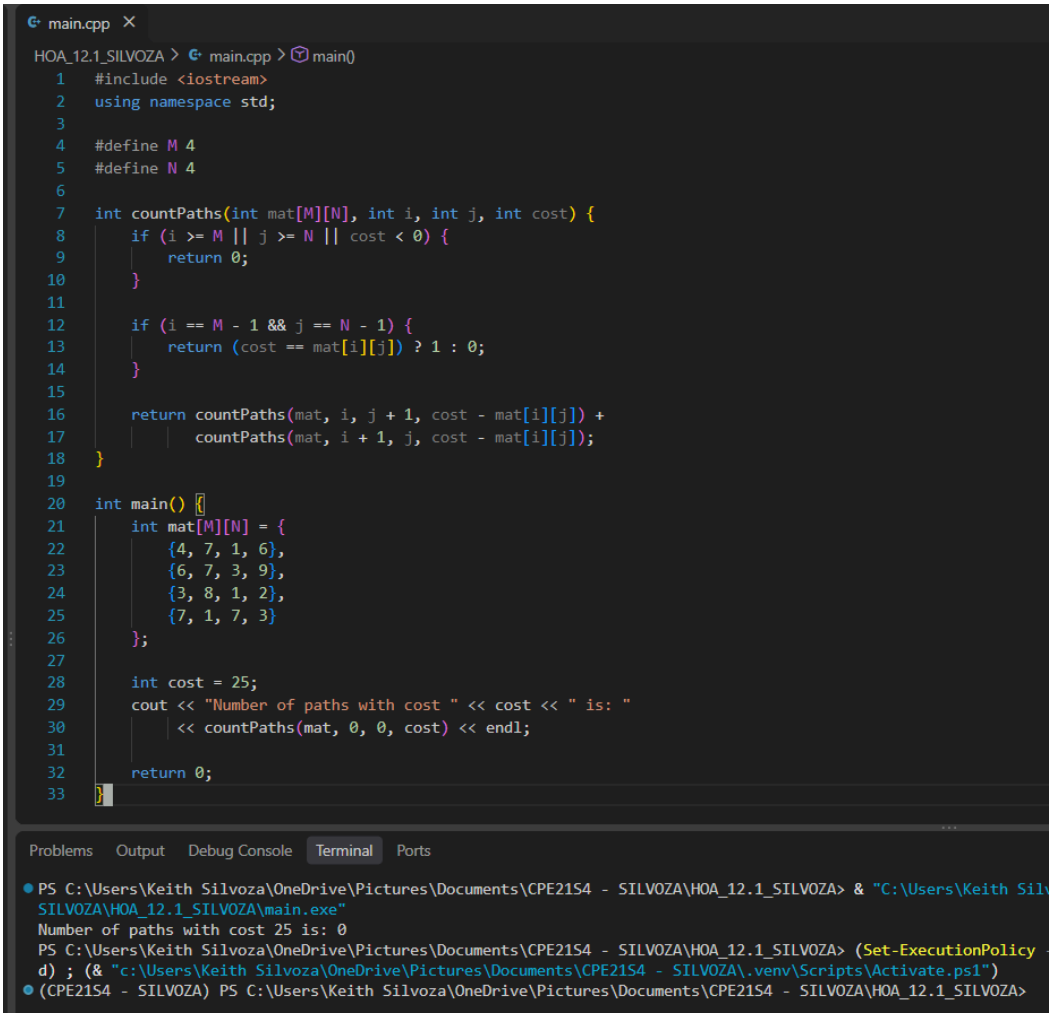
    IF i == last_row AND j == last_col THEN
        IF cost == matrix[i][j] THEN
            RETURN 1
        ELSE
            RETURN 0

    return countPaths(matrix, i, j + 1, cost - matrix[i][j]) +
           countPaths(matrix, i + 1, j, cost - matrix[i][j])
```

Given matrix:

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

Target Cost = 25
Solved by hand
$(0, 0) \rightarrow 4$
$(0, 1) \rightarrow 7$
$(0, 2) \rightarrow 1$
$(1, 2) \rightarrow 3$
$(2, 2) \rightarrow 1$
$(3, 2) \rightarrow 7$
$(3, 3) \rightarrow 3$
4+7+1+3+1+7+3=**26**



```cpp
#include <iostream>
using namespace std;

#define M 4
#define N 4

int countPaths(int mat[M][N], int i, int j, int cost) {
    if (i >= M || j >= N || cost < 0) {
        return 0;
    }

    if (i == M - 1 && j == N - 1) {
        return (cost == mat[i][j]) ? 1 : 0;
    }

    return countPaths(mat, i, j + 1, cost - mat[i][j]) +
           countPaths(mat, i + 1, j, cost - mat[i][j]);
}

int main() {
    int mat[M][N] = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int cost = 25;
    cout << "Number of paths with cost " << cost << " is: "
         << countPaths(mat, 0, 0, cost) << endl;

    return 0;
}
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\HOA_12.1_SILVOZA> & "C:\Users\Keith Silv
SILVOZA\HOA_12.1_SILVOZA\main.exe"
Number of paths with cost 25 is: 0
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\HOA_12.1_SILVOZA> (Set-ExecutionPolicy
d) ; (& "c:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\HOA_12.1_SILVOZA>
```

Analysis:

Recursive Path Finding: The algorithm moves **right** or **down** from cell $(0, 0)$, subtracting each cell's value from the target cost at every step.

Base:

- Returns 0 if it goes out of bounds or exceeds the cost limit.
- Returns 1 if it reaches the destination cell $(M - 1, N - 1)$ with the exact remaining cost

Efficiency: Plain recursion works for small grids but recalculates overlapping paths; dynamic programming (memoization) can be used to optimize execution speed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Code formatting, explanation of concepts, and grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to help troubleshoot C++ syntax errors, format report tables, and explain algorithmic concepts. All analysis, code execution, screenshot generation, and final submission reflect my own original work and understanding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.